

Indian Institute of Technology, Jodhpur

Department Of CSE

PCS II

Lab: Process Memory Information in xv6

Course Instructor: Dr Siddharth Sharma

Pre requisite-

- Make sure you have downloaded and installed the **xv6 operating system source code** provided for the lab.

The operating system runs on an x86 emulator called **QEMU**.

In the xv6 directory run the following commands:

```
make
make qemu
```

These commands compile xv6 and start the operating system inside the emulator.

- In this project you will explore how **memory is managed for a process in xv6**.

Before starting the implementation, you should study the following files in the xv6 source code.

```
proc.c
proc.h
vm.c
mmu.h
syscall.c
sysproc.c
user.h
usys.S
```

- **proc.h** contains the definition of the **process structure**.
- **proc.c** contains process related functions.
- **vm.c** contains functions related to **virtual memory management**.
- **mmu.h** defines page table related constants and structures.

-**syscall.c** and **sysproc.c** implement system calls.

Understanding these files will help you understand how xv6 manages **process address spaces and page tables**.

- Learn how to write and run **user programs in xv6**.

User programs must be added to the **Makefile** so that they can be compiled and executed inside xv6.

Remember that xv6 does not contain a compiler inside the operating system.
All programs must be written and compiled **before running xv6**.

Part A: Understanding Process Memory in xv6

In this part you will explore how xv6 organizes the memory of a process.

Each process in xv6 has its own **virtual address space** that contains different segments such as code, data, heap, and stack.

Tasks:

1. Locate the definition of struct proc in **proc.h**.
2. Identify the field that stores the **size of the process memory**.
3. Find where xv6 **allocates memory to a process** when it grows.
4. Study the functions in **vm.c** that manage page tables.
5. Observe how xv6 maps **virtual addresses to physical memory pages**.

Write a short user program that allocates memory and observe how the process memory size changes.

Part B: Implementing Memory Information System Calls

In this part you will extend xv6 by implementing system calls that provide **memory information of a process**.

The goal is to allow a user program to obtain information about the memory used by the process.

Tasks:

1. Implement a system call that returns the **total size of memory used by the current process**.
2. Implement a system call that returns the **number of virtual pages used by the process**.
3. Implement a system call that returns the **number of valid entries in the page table**.

For each system call you must:

- define a syscall number
- add the syscall to the syscall table
- implement the kernel function
- provide a user level interface

Part C: Writing a User Program

Write a user program that calls the system calls implemented above and prints the memory information of the current process.

The program should display information such as:

```
Process Memory Size : <value>
Virtual Pages Used : <value>
Page Table Entries : <value>
```

Add this program to the **Makefile** so that it is compiled along with other user programs.

Run the program inside xv6 and observe the output.

Hints

- Look at the implementation of existing system calls such as **getpid()** to understand how new system calls are added.
- The current process can be obtained using the **myproc()** function.
- The process structure contains useful information about the process memory.
- Page size in xv6 is defined by the constant **PGSIZE**.

- Use **cprintf()** if you want to print debugging messages from the kernel.

Testing

After implementing the system calls, test them using different scenarios.

You may try the following experiments:

1. Run the program immediately after system startup and observe the memory usage.
2. Modify the user program so that it allocates additional memory and check how the values change.
3. Create a child process using **fork()** and observe the memory information of the parent and child processes.

Verify that the values returned by the system calls are consistent with the expected behavior of the system.